

HelixPlay

HelixPlay

Пераўтварце любую машыну з GPU у ўласны прылада для воблачнага геймінгу.

Змест

HelixPlay — гэта самастойна разгортвальная платформа для воблачнага геймінгу, якая пераўтварвае любую машыну з падтрымкай GPU у адалены стрымінгавы сервер, забяспечваючы гульнявы досвед кансольнага ўзроўню на дэсктопных, мабільных, тэлевізійных і браўзерных кліентах. Яна створана як манарэпазіторый на аснове Go з 46 падмадулямі і трохслаёвым кліенцкім стэкам, а таксама можа быць перабрендавана для партнёраў.

Кароткае апісанне

HelixPlay — гэта самастойна разгортвальная, адкрытая і перабрендаваная платформа для воблачнага геймінгу. Яна пераўтварвае любую машыну з падтрымкай GPU у адалены стрымінгавы сервер і забяспечвае гульнявы досвед кансольнага ўзроўню на дэсктопных, мабільных, тэлевізійных і браўзерных кліентах праз WebRTC/QUIC з ядром на Go і кліенцкім стэкам Wails/Flutter/Angular.

Падрабязнае апісанне

HelixPlay — гэта платформа для воблачнага геймінгу, створаная як манарэпазіторый на аснове Go, які складаецца з 46 Git-падмадуляў. Яна пераўтварвае любую вашу ўжо наяўную гульнявую ПК у стрымінгавы сервер, забяспечваючы досвед кансольнага ўзроўню на дэсктопных, мабільных, тэлевізійных і браўзерных кліентах — самастойна

разгортвальная, адкрытая і прыдатная для перабрендавання партнёрамі. Ідэя простая: ваша апаратнае забеспячэнне, ваш сэрвіс, ваш брэнд, без пасярэднікаў у выглядзе трэціх бакоў.

Ключавым архітэктурным рашэннем з'яўляецца канвергенцыя трохслаёвага кліенцкага стэку — смелы тэхнічны выбар, які акупляецца ўсім астатнім. Дэсктопная праграма на Wails, мабільны/тэлевізійны дадатак на Flutter і вэб-кліент на Angular працуюць на аснове адзінага ядра Go, якое кампілюецца ў WASM для браўзера, таму паводзіны пішуцца адзін раз і выкарыстоўваюцца на ўсіх платформах замест таго, каб дублявацца ў трох варыянтах. Пад гэтым знаходзіцца шлях рэальнага часу для медыя: Захоп → Кадзіраванне → Пакетаванне → Перадача → Дэкадзіраванне → Адлюстраванне, які звязаны з платформа-спецыфічным захопам (DXGI / ScreenCaptureKit / PipeWire) і апаратнымі энкодэрамі (NVENC / QSV / AMF / VideoToolbox), каб GPU выконвала цяжкую працу, а перадача ажыццяўляецца праз WebRTC (Pion v4), QUIC (quic-go) і спецыяльныя UDP-датаграмы, абраныя з акцэнтам на нізкую латэнтнасць, а не на зручнасць. Базавае ядро кіруе сеансамі, кліентамі, каталогам і аўтэнтыфікацыяй; агенты на хосце адказваюць за захоп, кадзіраванне і перадачу на краі сеткі; а mDNS і рандэву-пратакты забяспечваюць аўтаматычнае выяўленне, каб кліенты знаходзілі свой сервер без ручной наладкі.

HelixPlay створана з нуля для перабрендаванага SaaS: тэматызацыя для кожнага кліента, фільтрацыя каталога, OAuth2 і білінг, каб партнёр мог запусціць поўнацэнны брэндаваны сэрвіс, а не проста перафарбаваць інтэрфейс. І яна цалкам кантэйнерызаваная: усе сэрвісы, базы даных, зборкі, тэсты і сканаванні працуюць у кантэйнерах, што робіць усю платформу паўторна разгортвальнай і праверанай. Як і ўвесь сямейны праект Helix, яна кіруецца анты-блефавай канстытуцыяй, дзе зялёны тэст гарантуе рэальную, прыдатную для канчатковага карыстальніка паводзіну — а не проста праходжанне макета.

Чаму мы яе стварылі

Камерцыйны воблачны геймінг закрыты, цэнтралізаваны і даступны толькі ў арэнду. HelixPlay створана для таго, каб кожны, хто мае машыну з GPU, мог запусціць уласны стрымінгавы сервер — адкрыты, самастойна разгортвальны і прыдатны для перабрендавання — замест таго, каб залежаць ад трэціх бакоў.

Змест

Чаму гэта змяняе правілы гульні

Тут аб'яднаны тры рэчы, якія камерцыйныя сэрвісы трымаюць паасобку: самастойны хостынг на абсталяванні пад вашым кантролем, адзінае ядро Go, якое кіруе трыма кліенцкімі стэкамі, каб функцыі з'яўляліся паўсюль адначасова, і шматкарыстальніцкі белы брэндынг. У выніку партнёр можа запусціць поўнаватрасны брэндаваны сэрвіс воблачнага геймінгу на сваіх *уласных* GPU — валодаючы досведам, карыстальнікамі і эканамічнымі паказчыкамі — замест таго, каб перапрадаваць магутнасці ў чужым воблаку і жыць у яго межах.

Што новага

- **Збег трох кліенцкіх стэкаў** — Wails, Flutter і Angular працуюць на адным ядры Go (WASM у браўзэры), таму дэсктоп, мабільныя прылады, тэлевізары і вэб карыстаюцца адной рэалізацыяй замест трох, якія разыходзяцца.
- **Самахостынг і белы брэндынг як SaaS** — убудаваныя магчымасці тэматызацыі для кожнага кліента, фільтрацыі каталогаў, OAuth2 і білінгу, таму платформа пастаўляецца як брэндаваны прадукт, а не дэмаверсія.
- **Сучасны транспарт з нізкай затрымкай** — WebRTC (Pion), QUIC і ўласны UDP у спалучэнні з аўтаматычным выбарам апаратнага энкодэра для кожнай платформы (NVENC / QSV / AMF / VideoToolbox), настроеныя на рэактыўнасць, а не на зручнасць.
- **Архітэктара з 46 дэкапліраванымі модулямі** — чыста аддзеленыя кампаненты з кантэйнераўтварэннем на ўсіх узроўнях: кожны сэрвіс, база даных, зборка, тэст і сканаванне працуюць у кантэйнерах.

Найбольшыя тэхнічныя выклікі і як мы іх пераадолелі

- **Стрымінг з нізкай затрымкай на разнародным абсталяванні.** Кожная AC і GPU па-рознаму рэалізуе захоп і энкодзінг, а затрымка не даравала памылак. Вырашана праз платформаазнаўчы шлях захопу/энкодзінгу — DXGI / ScreenCaptureKit / PipeWire, якія сілкуюць NVENC / QSV / AMF / VideoToolbox, перадаючы даныя праз WebRTC / QUIC / UDP, каб кожная машына выкарыстоўвала найхутчэйшы натыўны шлях да пікселяў.
- **Адзін прадукт для дэсктопа, мабільных прылад, тэлевізараў і вэбу.** Вырашана праз тры кліенцкія стэкі (Wails, Flutter, Angular), якія падзяляюць адзінае ядро Go, скампіляванае ў WASM для браўзэра, каб выпраўленне ці

новая функцыя, напісаная адзін раз, з'яўлялася на ўсіх чатырох платформах, а не пераносілася чатыры разы.

- **Шматкарыстальніцкі белы брэндынг.** Вырашана шляхам убудавання тэматызацыі для кожнага кліента, фільтрацыі каталогаў, OAuth2 і білінгу непасрэдна ў асноўны бэкэнд, каб ізаляцыя кліентаў і брэндынг былі прымітывамі платформы, а не індывідуальнымі галінамі для кожнага заказчыка.

Тэхнічны стэк

- **Go (1.26.2 root / 1.25+ submodules)** — агульнае ядро бэкэнда і хост-агент; адна мова, якая кампілюецца ў натывуныя бінарнікі і ў WASM, што робіць магчымай канцэпцыю адзінага ядра для некалькіх кліентаў.
- **Wails v2** — дэсктопны кліент, які звязвае ядро Go з убудаваным вэб-прэзентам, каб дэсктопная праграма выкарыстоўвала лагіку ядра непасрэдна, а не перапісвала яе.
- **Flutter 3.29+** — кліент для мабільных прылад і тэлевізараў, які звяртаецца да ядра Go праз FFI для натывунага інтэрфейсу на тэлефонах і тэлевізарах без дадатковага бэкэнда.
- **Angular 17+** — вэб-кліент, які запускаяе тое ж самае ядро Go, скампіляванае ў WASM, каб браўзер быў поўнаватэраснай платформай, а не спрошчанай версіяй.
- **WebRTC / Pion v4, QUIC / quic-go, уласны UDP** — тры транспарты рэальнага часу, каб платформа магла выбіраць шлях з найменшай затрымкай для кожнай сеткі і кліента.
- **Апаратныя энкодэры (NVENC / QSV / AMF / VideoToolbox) і захоп платформы (DXGI / ScreenCaptureKit / PipeWire)** — апаратна-паскораны шлях захопу і энкодзінгу GPU, які выбіраецца для кожнай платформы, каб энкодзінг ніколі не стаў вузкім месцам на ўзроўні CPU.
- **Кантэйнераўтварэнне (Docker/Podman)** — кожны сэрвіс, база даных, зборка, тэст і сканаванне працуюць у кантэйнерах, што робіць усю сістэму паўторна ўзнаўляльнай для разгортвання і праверкі.
- **mDNS / rendezvous** — аўтаматычнае выяўленне хоста без канфігурацыі, каб кліенты самастойна знаходзілі стрымінгавы сервер у сетцы.

Змест

Заўвагі аб статусе і сумленнасці

- **Статус: бэта-версія.** Мэты README па затрымках (≤ 30 мс у лакальнай сетцы / ≤ 50 мс у глабальнай сетцы р999),

фармулёўкі «клас кансолі / клас PS4 Pro» і колькасць ячэек тэставага набору з'яўляюцца дэклараванымі мэтамі праекта, якія не прайшлі незалежнага тэсціравання і падаюцца як такія.

- **Ліцэнзія: пакуль не вызначана.** Ліцэнзія не была выяўлена праз GitHub API — НЕПРАВЕРАНА / не дэкларавана.

Прыярытэтны ўзровень: Helix-асноўны.